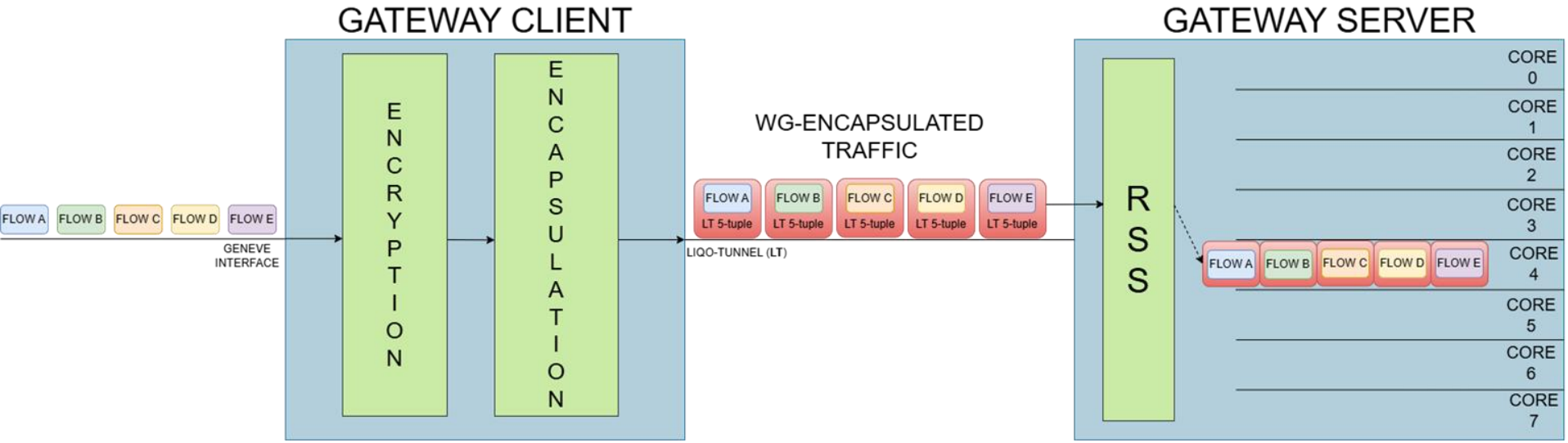
MULTI-TUNNEL in LIQO

maintainers’ meeting 09/03/2026

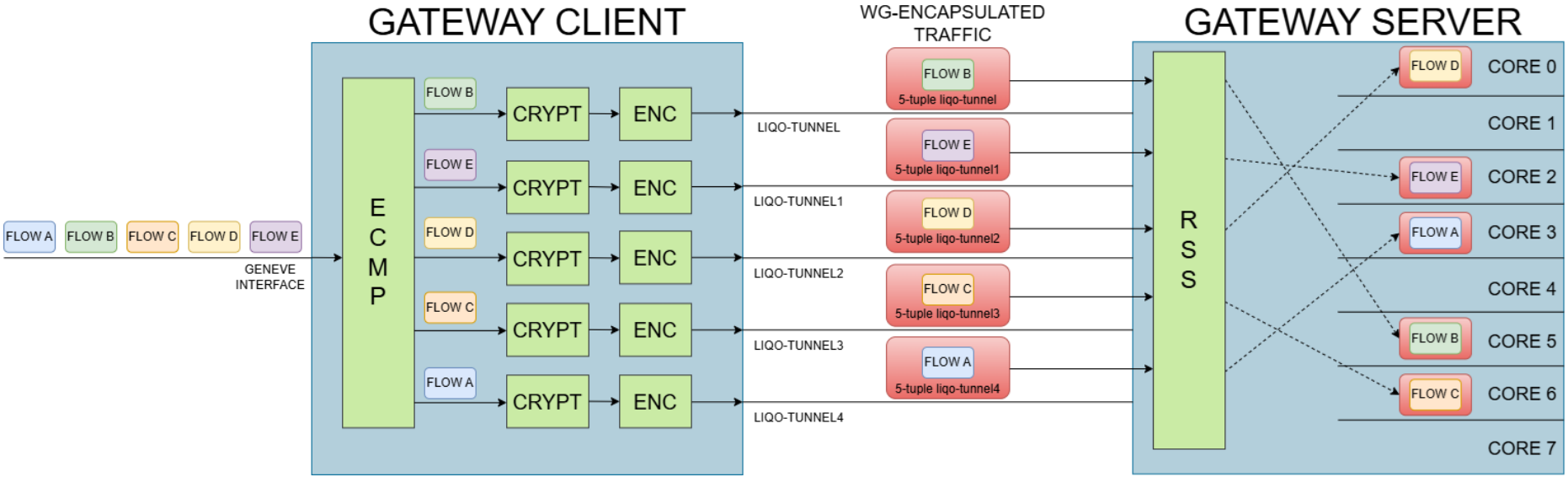
Motivation

- Liqo is reported to be slow when multiple flows run between two peered clusters
  - Example: https://github.com/liqotech/liqo/issues/3079
- **Cause:** each flow encapsulated by WireGuard shares the same 5-tuple and is processed by the same CPU core on reception
- Existing proposals to address the problem in WireGuard are still in the research phase and require kernel recompilation
  - Example: [REFS] https://doi.org/10.1093/comjnl/bxaf059
- **Simpler solution:** parallelize traffic with multiple WireGuard interfaces and distribute it using ECMP

Problem


GATEWAY CLIENT
GATEWAY SERVER
FLOW A
FLOW B
FLOW C
FLOW D
FLOW E
GENEVE INTERFACE
ENCRYPTION
ENCAPSULATION
WG-ENCAPSULATED TRAFFIC
LT 5-tuple
LIQO-TUNNEL (LT)
RSS
CORE 0
CORE 1
CORE 2
CORE 3
CORE 4
CORE 5
CORE 6
CORE 7

Solution


GATEWAY CLIENT
WG-ENCAPSULATED TRAFFIC
GATEWAY SERVER
FLOW A
FLOW B
FLOW C
FLOW D
FLOW E
GENEVE INTERFACE
ECMP
FLOW B
FLOW E
FLOW D
FLOW C
FLOW A
CRYPT
ENC
LIQO-TUNNEL
LIQO-TUNNEL1
LIQO-TUNNEL2
LIQO-TUNNEL3
LIQO-TUNNEL4
FLOW B
5-tuple liqo-tunnel
FLOW E
5-tuple liqo-tunnel1
FLOW D
5-tuple liqo-tunnel2
FLOW C
5-tuple liqo-tunnel3
FLOW A
5-tuple liqo-tunnel4
RSS
FLOW D
CORE 0
CORE 1
FLOW E
CORE 2
FLOW A
CORE 3
CORE 4
FLOW B
CORE 5
FLOW C
CORE 6
CORE 7

Preliminary Tests - Manual Implementation

- Single Machine, Quick-start setup
  - Single machine Cloudlab sm220u (32 cores), with the Quick-start setup (2 KinD clusters connected with Liqo)
- Peering only (No Offloading)
- Pod-to-Pod tests: iperf3 between two nginx pods (average of 5 runs)

	Baseline Aggregated Throughput (Gbit/s)	2 tunnels Aggregated Throughput (Gbit/s)	4 tunnels Aggregated Throughput (Gbit/s)
1 iperf3 Flow	1.26	1.29	1.28
2 iperf3 Flows	1.24	1.28	1.97
4 iperf3 Flows	1.24	2.43	2.87
8 iperf3 Flows	1.25	2.39	4.11
16 iperf3 Flows	1.24	2.42	4.45

Final Results - Automated Implementation

Tests conducted under the same conditions as the preliminary phase

	1 Interface		2 Interfaces		4 Interfaces		8 Interfaces	
	Th(Gbit/s)	% CPU	Th(Gbit/s)	% CPU	Th(Gbit/s)	% CPU	Th(Gbit/s)	% CPU
1 Flow	1,32	15,61	1,32	15,68	1,33	15,21	1,34	15,11
2 Flow	1,31	15,61	2,07	23,66	1,84	21,03	2,59	30,31
4 Flow	1,30	15,07	2,31	26,9	3,29	33,88	4,19	47,54
8 Flow	1,31	15,62	2,57	29,88	4,31	40,54	5,98	64,02
16 Flow	1,29	15,6	2,52	29,29	4,63	48	7,55	73,94
32 Flow	1,28	16,17	2,50	29,74	4,82	52,05	8,75	79,06
64 Flow	1,25	17,54	2,41	30,94	4,69	52,23	8,71	80,03
128 Flow	1,28	17,96	2,33	32,8	4,48	55,03	8,51	81,09
	16 Interfaces		**32 Interfaces**		**64 Interfaces**		**128 Interfaces**	
	Th(Gbit/s)	% CPU	Th(Gbit/s)	% CPU	Th(Gbit/s)	% CPU	Th(Gbit/s)	% CPU
1 Flow	1,30	15,69	1,23	15,19	1,08	14,91	0,86	14,65
2 Flow	2,37	28,43	2,24	26,16	1,88	24,58	1,43	21,65
4 Flow	3,90	44,61	4,16	47,52	3,93	46,76	2,80	45,87
8 Flow	6,84	71,78	7,27	73,02	6,07	70,21	4,69	67,75
16 Flow	9,37	82,80	10,44	86,89	8,66	84,00	7,33	85,86
32 Flow	11,17	88,73	13,14	92,74	12,02	93,15	10,42	93,59
64 Flow	12,72	91,77	15,82	95,95	14,55	95,86	12,01	96,17
128 Flow	14,29	93,10	19,12	97,34	17,59	97,28	15,94	97,38

Implementation: High-Level Steps

1. Modify the gateway to create and configure an arbitrary number of Wireguard interfaces
   a. The existing keypair can be reused for all interfaces
   b. Server interfaces can be exposed by adding their ports to the existing service
   c. Only the number of desired interfaces and their associated ports (listenports, endpointports) need to be declared in the *GatewayClient* and *GatewayServer* CRD
2. Extend routes and NAT rules inside the gateway
   a. Update RouteConfigurations *gw-ext* to include the new interfaces (next-hops)
   b. ECMP is the simpler choice to load balance the traffic
   c. Other Route and Firewall configurations involving WireGuard interfaces are left unchanged: they are expanded internally by the gateway based on the interface count

More details in the PoC:
https://github.com/MircoBarone/liqo/tree/multi-tunnel

IMPLEMENTAZIONE

Parametri aggiunti

Container Gateway:

--enable-ecmp

Container Wireguard:

--interface-number

--endpoint-port (Supporto esteso a N porte sul client)

--listen-port (Supporto esteso a N porte sul server)

GatewayClient /Gateway Server

Commit : `f4eb8d7, 97cd596, 224eb10, 4247b3c`

GatewayClient

- Aggiunto il campo **endpoint.otherPorts** (*struct EndpointStatus*) che riporta le porte utilizzate dal Service per esporre le interfacce remote sul server
  - *Il campo endpoint.port viene mantenuto per retrocompatibilità*
  - Tramite un opportuno template i valori possono essere accodati nel parametro *--endpoint-port* del container wireguard che li utilizzerà per configurare gli endpoint delle interfacce del client
- Aggiunto un parametro numInterfaces (default=1) che indica il numero di interfacce da creare
  - *Viene scritto nel campo numInterfaces della crd WgGatewayClient (opportunamente estesa)*
  - Viene passato (tramite template) al comando *interface-number* (wireguard container) e se >1 il comando *enable-ecmp* si setta a true (gateway container)

```
  endpoint:
    addresses:
    - 172.18.0.4
    otherPorts:
    - 30440
    - 32276
    - 32220
    port: 32203
    protocol: UDP
  mtu: 1340
  numInterfaces: 4
  secretRef:
    name: ""
```

GatewayServer

- Aggiunto alla spec il campo **endpoint.otherPorts** (struct *Endpoint*) che contiene le listening ports delle interfacce create
  - *Il campo endpoint.port viene mantenuto per retrocompatibilità*
  - Tramite un opportuno template i valori possono essere accodati nel comando --listen-port del container wireguard
- Aggiunto il parametro **numInterfaces** gestito in modo analogo al Client

Gateway Server problemi

- Se numInterfaces=otherPorts+1 tutto ok
- Se numInterfaces < otherPorts+1 le porte in eccesso sono ignorate
- Se numInterfaces > otherPorts +1 si creano delle interfacce senza porte. Queste non vanno mai up, è un problema anche abilitare la threaded Napi
- Il vincolo in questione non può essere imposto direttamente sullo yaml

- **Soluzioni**:
  - **Controllo del vincolo alla creazione di wggatewayserver**:
    - Se ci sono porte in eccesso vengono rimosse da gatewayserver. Se mancano porte si aggiungono in modo casuale a gatewayserver, evitando conflitti con quelle già esistenti. La risorsa modificata fa ripartire il Reconciler
  - Utilizzo di un webhook per validare la risorsa – potrebbe essere complesso da implementare
  - Eliminazione del campo numInterfaces, derivato dal numero di porte direttamente - poco user friendly
  - Calcolo dinamico delle porte mancanti, al momento della creazione del gateway - problematico perché in gatewayserver non vi è traccia delle porte aggiunte. Problemi nella creazione del service

Gateway Client problemi e next-steps

- otherPorts vengono copiate direttamente da quelle che espone il gatewayserver
- La soluzione semplice è eliminare il campo numInterfaces

Per evitare di utilizzare somme nel template è meglio fare un refactoring più complesso eliminando la dualità tra port e otherPorts sia sul client che sul server

Creazione e configurazione interfacce

Commit : `27b7a1a, 61d780d, 33fdda2`

Creazione di N interfacce

- In *cmd/gateway/wireguard/main.go* : **InitWireguardLink** è chiamata ripetutamente in un loop (con limite *--interface-number*).
  Ad ogni chiamata si passa l'indice del loop alla funzione
- La funzione genera:
  - Il **nome** che diventa <nome di base>-k (eccetto che per k=0) (es *liqo-tunnel*, *liqo-tunnel1*)
  - L'**ip**: si costruisce partendo dagli ip di base (169.254.18.2 e 169.254.18.1) e aggiungendo 4 (rete /30)
  - La **listening port** (solo per il gateway server) prendendola da *--listen-port*

Configurazione di N interfacce

- Le interfacce create al punto precedente hanno solo nome e listening port. Necessari ancora i parametri:
  - Chiave privata
  - Chiave pubblica del peer
  - Allowed-ips
  - Endpoint port (Solo per il client) L'indirizzo e la porta da utilizzare per contattare il peer
- Patch della funzione **configureDevice** (*pkg/gateway/tunnel/wireguard/device.go*), chiamata in un loop nel Reconcile (pkg/gateway/tunnel/wireguard/publickeys_controller.go) :
  - Introduzione di un parametro intero idx: utilizzato per identificare l'interfaccia corretta in base al nome, ricalcolare la listening port lato server, selezionare l'EndpointPort lato client (dalla lista *--endpoint-port*)
  - EndpointIP (client) e allowed-ips sono uguali per tutte le interfacce.
- Una singola coppia di chiavi é sufficiente per tutte le interfacce

Gestione di Service,Handshake e Status

Commit : `4778d10, ff59ba3, fbbc5ed, 71167a7`

Gestione del Service

- Il service che espone le interfacce del Server viene creato automaticamente da Kubernetes sulla base della risorsa *wggatewayserver* che ne contiene il deployment completo (generato a partire dal template)
- **Idea:** estendere il template in modo che si crei un unico service che rimappi tutte le listening ports (*otherPorts*) contenute nella risorsa GatewayServer **Problema:**Il template non può essere modificato per inserire un numero non definito di porte al suo interno (ad esempio con un ciclo)
- **Approccio scelto:** estendere il GatewayServer controller in modo che, lette le porte dal GatewayServer, le inserisca direttamente nella spec del Service in *wggatewayserver*
- **Nodeport port**: Il campo Nodeport del gatewayServer non è stato ancora esteso per contenere N porte. Una volta fatto queste dovranno essere integrate nel service (quando specificate)

```
spec:
  ports:
  - port: 51840
    protocol: UDP
    targetPort: 51840
  selector:
    app.kubernetes.io/component: networking
    app.kubernetes.io/instance: liqo-rome
    app.kubernetes.io/managed-by: Helm
    app.kubernetes.io/name: rome
    app.kubernetes.io/part-of: liqo
    networking.liqo.io/active: "true"
    networking.liqo.io/component: gateway
    networking.liqo.io/gateway-name: rome
    networking.liqo.io/gateway-namespace: liqo-tenant-rome
  type: NodePort
```

```
spec:
  ports:
  - name: liqo-tunnel
    port: 51840
    protocol: UDP
    targetPort: 51840
  - name: liqo-tunnel-1
    port: 51940
    protocol: UDP
    targetPort: 51940
  - name: liqo-tunnel-2
    port: 52040
    protocol: UDP
    targetPort: 52040
  - name: liqo-tunnel-3
    port: 52140
    protocol: UDP
    targetPort: 52140
  - name: liqo-tunnel-4
    port: 52240
    protocol: UDP
    targetPort: 52240
```

Service Status

- Dopo aver creato il Service multiporta è necessario allineare le porte effettive nello status delle CRD *WgGatewayServer* e *GatewayServer*
  - Si può sfruttare il campo *otherPorts* della struct EndpointStatus accodandoci le porte aggiuntive presenti nel Service
    - Piccola modifica delle funzioni *forgeEndpointStatusClusterIp*, *forgeEndpointStatusNodePort*, *forgeEndpointStatusLoadBalancer*
  - L'utilizzo di un unico Service consente di gestire solo la scrittura del campo otherPorts lasciando inalterata la gestione del ciclo di vita dell'Endpoint
- Una volta scritto lo status di *WgGatewayServer* questo viene letto e copiato in *GatewayServer*. Richiede una piccola modifica alla funzione ParseEndpoint

Keepalive

- Liqo utilizza un sistema di keepalive basato su un ping-pong ICMP
  - Serve per calcolare la latenza della connessione e per aggiornare lo stato della stessa
  - Tale sistema è stato lasciato inalterato per l’interfaccia liqo-tunnel
- Su tutte le interfacce aggiuntive è stato utilizzato il persistent keepalive nativo
  - Valore di 25 secondi hardcoded
  - Ha il solo scopo di mantenere le interfacce connesse e mantenere attive le regole di nat intermedie.
    - Altrimenti il ping server-client sarebbe impossibile (Ad esempio il server impara l’endpoint del client solo dopo la prima connessione)

Threaded NAPI

Commit : `756567c`

Threaded Napi

- L’abilitazione della threaded NAPI su Wireguard avviene unicamente settando ad 1 */sys/class/net/<interface>/threaded*
  - Non esistono API netlink o syscall dedicate
- **Problema:** il container gateway, pur essendo privilegiato, opera con /sys montato in read-only
- Due soluzioni:
  - **Il container gateway attende la creazione delle interfacce da parte del container Wireguard. Esegue il remount di /sys in modalità read-write. Aggiorna il flag di tutte le interfacce. Ripristina /sys in modalità read-only**
    - Si fa tutto tramite una go routine dentro il metodo Start() del RunnableGateway
  - Il container wireguard, reso privilegiato, setta il flag subito dopo la creazione delle interfacce. E’ necessario comunque fare il mounting in read-write di sys
- Possibili problemi (soluzione 1):
  - Lettura di un numero di interfacce inferiore a quello reale
  - Tentativo di abilitare il flag se non supportato - loop infinito

RouteConfiguration

Commit : `e2c94ce ,750e07d, 56f87f3, 5a42702`

<remote-cluster-id>-gw-ext

- Aggiunto il supporto ad una struttura (retrocompatibile) **next-hop** nel file **apis/networking/v1beta1/routeconfiguration_types.go**
  - Next-Hop contiene un campo gateway (indirizzo IP del next-hop) e un campo weight (peso ECMP della rotta)
  - Un vettore di Next-Hop (*nextHops*) può essere utilizzato come alternativa (mutuamente esclusiva) al campo gateway
- La funzione *forgeNetlinkRoute* (file *liqo/pkg/route.go*) gestisce il multipath
  - Si utilizza il campo *MultiPath* di *netlink.route* insieme alla struttura *netlink.NexthopInfo*
  - Funzione isEqualRoute è estesa il confronto tra route Multipath e singole
    - Il confronto tra rotte singole rimane invariato

```yaml
- dst: 10.200.0.0/16
  iif: liqo.k9qp55kvct
  routes:
  - dst: 10.200.0.0/16
    nextHops:
    - gw: 169.254.18.5
      weight: 1
    - gw: 169.254.18.1
      weight: 1
```

<remote-cluster-id>-gw-ext ECMP

- Nel main del container gateway il valore del parametro *--enable-ecmp* si riflette nel parametro *net.ipv4.fib_multipath_hash_policy* all'interno del pod gateway, ponendolo uguale a 1 (se true) al fine di supportare l'hash ECMP sulla 5-tupla.
  - Il parametro è per namespace: se modificato nel gateway non si riflette sul nodo host
- Il valore di *--enable-ecmp* dipende dal numero di interfacce:
  - TRUE se le interfacce sono almeno 2

<remote-cluster-id>-gw-ext

- Nella funzione **enforceRouteConfigurationPresence (***Configuration Controller del Controller Manager***)**:
  - Estrazione dinamica del numero di interfacce dal campo numInterfaces della CRD *GatewayClient*/*GatewayServer*
  - Calcolo degli gli IP remoti di tutte le interfacce del gateway
  - Entrambi i parametri passati alla funzione **forgeMutateRouteConfiguration**: ora genera condizionalmente rotte standard (gw) o rotte multipath (nextHops) in base al numero di interfacce
- Limitazioni attuali:
  - Non è stato gestito il caso in cui una delle interfacce perda la connessione
  - Il numero di interfacce è letto dallo Spec delle CRD *GatewayClient* /*Gateway Server* perchè non esiste (ancora) un equivalente nello Status

milan-gw-ext ESEMPIO

```yaml
table:
  name: milan
  rules:
  - dst: 10.200.0.0/16
    iif: liqo.v8cvgnhrc5
    routes:
    - dst: 10.200.0.0/16
      nextHops:
      - gw: 169.254.18.1
        weight: 1
      - gw: 169.254.18.5
        weight: 1
      - gw: 169.254.18.9
        weight: 1
      - gw: 169.254.18.13
        weight: 1
      - gw: 169.254.18.17
        weight: 1
      - gw: 169.254.18.21
        weight: 1
  - dst: 10.70.0.0/16
    iif: liqo.v8cvgnhrc5
    routes:
    - dst: 10.70.0.0/16
      nextHops:
      - gw: 169.254.18.1
        weight: 1
      - gw: 169.254.18.5
        weight: 1
      - gw: 169.254.18.9
        weight: 1
      - gw: 169.254.18.13
        weight: 1
      - gw: 169.254.18.17
        weight: 1
      - gw: 169.254.18.21
        weight: 1
```

```yaml
  - dst: 10.200.0.0/16
    iif: liqo.fcgcsdpbdn
    routes:
    - dst: 10.200.0.0/16
      nextHops:
      - gw: 169.254.18.1
        weight: 1
      - gw: 169.254.18.5
        weight: 1
      - gw: 169.254.18.9
        weight: 1
      - gw: 169.254.18.13
        weight: 1
      - gw: 169.254.18.17
        weight: 1
      - gw: 169.254.18.21
        weight: 1
  - dst: 10.70.0.0/16
    iif: liqo.fcgcsdpbdn
    routes:
    - dst: 10.70.0.0/16
      nextHops:
      - gw: 169.254.18.1
        weight: 1
      - gw: 169.254.18.5
        weight: 1
      - gw: 169.254.18.9
        weight: 1
      - gw: 169.254.18.13
        weight: 1
      - gw: 169.254.18.17
        weight: 1
      - gw: 169.254.18.21
        weight: 1
```

```
32761:  from all to 10.70.0.0/16 iif liqo.fcgcsdpbdn lookup milan
32762:  from all to 10.200.0.0/16 iif liqo.fcgcsdpbdn lookup milan
32763:  from all to 10.70.0.0/16 iif liqo.v8cvgnhrc5 lookup milan
32764:  from all to 10.200.0.0/16 iif liqo.v8cvgnhrc5 lookup milan
```

```
10.70.0.0/16
        nexthop via 169.254.18.1 dev liqo-tunnel weight 1
        nexthop via 169.254.18.5 dev liqo-tunnel1 weight 1
        nexthop via 169.254.18.9 dev liqo-tunnel2 weight 1
        nexthop via 169.254.18.13 dev liqo-tunnel3 weight 1
        nexthop via 169.254.18.17 dev liqo-tunnel4 weight 1
        nexthop via 169.254.18.21 dev liqo-tunnel5 weight 1
10.200.0.0/16
        nexthop via 169.254.18.1 dev liqo-tunnel weight 1
        nexthop via 169.254.18.5 dev liqo-tunnel1 weight 1
        nexthop via 169.254.18.9 dev liqo-tunnel2 weight 1
        nexthop via 169.254.18.13 dev liqo-tunnel3 weight 1
        nexthop via 169.254.18.17 dev liqo-tunnel4 weight 1
        nexthop via 169.254.18.21 dev liqo-tunnel5 weight 1
```

<local-cluster-id>-<node-name>-gw-node

- Questa Risorsa viene scritta nel momento in cui Liqo viene installato, quando ancora il peering non è avvenuto
  - Prevede un unico campo *iif: liqo-tunnel* (Il nome dell'interfaccia è hardcoded)
  - Viene aggiornata quando un pod viene aggiunto o rimosso da un nodo
  - Estenderla può essere complicato:
    - Richiede di reagire alla creazione di un nuovo gateway, e di calcolare il numero massimo di interfacce esistenti.
    - Richiederebbe uno yaml molto lungo, perchè per ogni interfaccia bisogna replicare le stesse regole
- **Soluzione:** Lasciare la CRD inalterata. Cambiare come viene interpretata dal controller delle rotte dentro al gateway.
  - Quando legge *iif: liqo-tunnel* controlla quante interfacce liqo-tunnel* esistono e replica internamente la regola per ciascuna delle interfacce

```yaml
spec:
  table:
    name: rome-control-plane
    rules:
    - dst: 10.80.0.1/32
      routes:
      - dev: liqo.x5mqpv69x4
        dst: 10.80.0.1/32
        scope: link
    - iif: liqo-tunnel
      routes:
      - dst: 10.200.0.3/32
        gw: 10.80.0.1
        targetRef:
          kind: Pod
          name: local-path-provisioner-988d74bc-bx5jw
          namespace: local-path-storage
          uid: 7a7dfa42-afaa-4edb-88cd-ad7af828f0af
      - dst: 10.200.0.2/32
        gw: 10.80.0.1
        targetRef:
          kind: Pod
          name: coredns-7db6d8ff4d-7ds82
          namespace: kube-system
          uid: 45bb1b07-0570-4b5f-a09a-13ec55b46015
      - dst: 10.200.0.4/32
        gw: 10.80.0.1
        targetRef:
          kind: Pod
          name: coredns-7db6d8ff4d-qs2tk
          namespace: kube-system
          uid: d345729a-0ae8-4ca6-8004-c7497017c9e7
```

<local-cluster-id>-<node-name>-gw-node ESEMPIO

Comando : *ip rule show all* sul cluster Rome

```
32747:  from all iif liqo-tunnel4 lookup rome-worker
32748:  from all iif liqo-tunnel3 lookup rome-worker
32749:  from all iif liqo-tunnel2 lookup rome-worker
32750:  from all iif liqo-tunnel1 lookup rome-worker
32751:  from all iif liqo-tunnel lookup rome-worker
32752:  from all iif liqo-tunnel4 lookup rome-control-plane
32753:  from all iif liqo-tunnel3 lookup rome-control-plane
32754:  from all iif liqo-tunnel2 lookup rome-control-plane
32755:  from all iif liqo-tunnel1 lookup rome-control-plane
32756:  from all iif liqo-tunnel lookup rome-control-plane
```

FirewallConfiguration

Commit : `4006e49, 199f843, 1bdb530, 80cdb09`

<tenant-name>-remap-podcidr /<tenant-name>-remap-externalcidr

1. Si tratta di due CRD che gestiscono il remapping. Vengono tradotte da un controller nel gateway in delle regole nftables
2. Il Reconciler associato si attiva ogni volta che la *CRD Configuration* subisce una modifica
   a. **Problema:** la CRD Configuration rimane invariata quando i gateway vengono ricreati (ad esempio con un numero diverso di interfacce) quindi lo yaml della *FirewallConfiguration* resta inalterato
3. Soluzioni:
   a. **Nessuna modifica al Reconciler della firewall configuration. Patch del controller del gateway in modo che consideri ogni occorrenza di “liqo-tunnel” un placeholder e lo specializzi con il numero di interfacce effettive**
   b. Aggiungere un watch al Reconciler per le risorse *GatewayClient*/*GatewayServer*
   c. Modificare la risorsa Configuration aggiungendo un campo che indichi il numero di interfacce

tenant-name>-remap-podcidr /<tenant-name>-remap-externalcidr DETTAGLI

- Si costruisce un set nftables che contiene tutte le interfacce
- Il traffico in uscita dal gateway necessita di subire DNAT. Nella regola base si specifica che il traffico non deve entrare dall'interfaccia "liqo-tunnel"
  - Dentro il gateway la regola si espande in modo da fare *NOT IN* nel set
- Il traffico in ingresso necessita di subire SNAT. Nella regola di base si specifica come interfaccia di ingresso "liqo-tunnel"
  - Dentro il gateway la regola si espande in modo da fare *IN* nel set
- La firewallconfiguration **service-nodeport-routing** marca il traffico che entra dall'interfaccia liqo-tunnel. La sintassi è molto simile al remapping
  - Anche in questo caso si è estesa la regola dentro il gateway per fare IN nel set

tenant-name>-remap-podcidr /<tenant-name>-remap-externalcidr ESEMPIO

```
table ip remap-podcidr {
        set tunnel-list-4 {
                type ifname
                elements = { "liqo-tunnel",
                             "liqo-tunnel1",
                             "liqo-tunnel2",
                             "liqo-tunnel3" }
        }

        chain outgoing {
                type nat hook prerouting priority dstnat; policy accept;
                ip daddr 10.61.0.0/16 iifname != "eth0" iifname != @tunnel-list-4 dnat prefix to 10.200.0.0/16 comment "80d3c648-8d48-41cb-b004-a88e9d9ba15b"
        }

        chain incoming {
                type nat hook postrouting priority srcnat; policy accept;
                oifname != "eth0" ip saddr 10.200.0.0/16 iifname @tunnel-list-4 snat prefix to 10.61.0.0/16 comment "d1aef0d0-f51a-435b-821a-01fbdd99a75c"
        }
}
```

```
table ip service-nodeport-routing {
        set tunnel-list-4 {
                type ifname
                elements = { "liqo-tunnel",
                             "liqo-tunnel1",
                             "liqo-tunnel2",
                             "liqo-tunnel3" }
        }

        chain mark-to-conntrack {
                type filter hook forward priority filter; policy accept;
                ip saddr 10.70.0.0 iifname "liqo.c8qjcbf25n" ct mark set 0x00000001 comment "rome-control-plane"
                ip saddr 10.70.0.0 iifname "liqo.pkbhlqfhq9" ct mark set 0x00000002 comment "rome-worker"
        }

        chain conntrack-mark-to-meta-mark {
                type filter hook prerouting priority filter; policy accept;
                ip daddr 10.70.0.0 iifname @tunnel-list-4 meta mark set ct mark comment "conntrack-mark-to-meta-mark"
        }
}
```

Next Steps

- Integrazione con Liqoctl
- Rimozione dualità port otherPorts
- Estensione Nodeport port